

Packaging basics

For years, packaging sounded boring – like the plastic around a Cadbury. Not anymore.

As front-end transistor scaling gets tougher, performance and efficiency gains increasingly come from how multiple dies are integrated – sometimes side-by-side, sometimes stacked.

You'll hear terms like:

- 2.5D packaging (dies placed on an interposer)
- 3D packaging (dies stacked, often with advanced bonding)
- chiplets (modular dies combined into one package)

Recent industry explainers highlight how 2.5D and 3D packaging approaches differ and why they're used.

And packaging guides emphasize that advanced packaging enables integrating multiple dies to improve bandwidth and power – especially as traditional scaling slows.

Chiplets: Lego for grown-ups

A chiplet is a small integrated circuit designed to be combined with other chiplets in a package to build a larger system. One big advantage is yield: smaller dies are easier to manufacture at high yield, and you can assemble “known good” chiplets into the final product.

This is why companies like AMD made chiplet architectures famous in CPUs, and Intel announced doing something similar with its Panther Lake chips, and why advanced packaging is now a strategic battleground alongside fabs.

Also, chiplets are increasingly supported by interconnect standards like UCIe, which aims to make die-to-die connectivity more interoperable.

Where chips fail, why testing is a career

Now the unglamorous truth: chips are hard to manufacture because physics and dust hate you.

In fab terms, yield is the percentage of usable chips you get out of what you *could* have produced on a wafer.

Higher yield means more sellable chips per wafer, which directly affects cost and supply.

Yield matters because defects happen:

- particles,
- process variation,
- tiny patterning errors,
- material imperfections.

And as chips get more complex, testing becomes a huge discipline. A chip isn't “done” when it's manufactured – it's done when it's verified, binned, and packaged reliably.

This is also why “known good die” matters in advanced packaging: you'd rather test dies first and combine working ones than assemble a mega-package and discover one block is dead.

Identify the chips inside your gadgets

This is your first “doer” exercise. You're going to learn how to look

20 TERMS RELATED TO CHIPS

Start getting comfortable hearing these terms:

- **Transistor / MOSFET:** the switch.
- **Gate / Source / Drain:** MOSFET terminals.
- **Logic gate:** AND/OR/NOT building blocks
- **Flip-flop:** stores 1 bit, two stable states.
- **Clock:** timing heartbeat for sequential logic
- **Setup/Hold time:** how long data must be stable around a clock edge.
- **Node (nm):** manufacturing generation label, not a single size.
- **Moore's Law:** transistor counts historically doubled about every two years.
- **Dennard scaling:** shrinking once helped keep power density stable.
- **Yield:** % of good chips per wafer.
- **Package:** the “home” of the die(s), now performance-critical.
- **2.5D/3D packaging:** multi-die integration approaches.
- **Chiplet:** modular die combined into a package.
- **Known good die (KGD):** tested dies used for higher overall yield.

at a device and stop seeing “mystery rectangles.” You'll start recognizing roles: power management, memory, radios, controllers.

Step-by-step

1. Pick a device you own: old smartphone, router, smartwatch, earbuds case, even a power bank.
2. Search for an iFixit teardown or board photos (model name + “teardown”).
3. Use iFixit's component identification guide to understand what's what on a typical device (battery, display, etc.).
4. For chips specifically, look for chip-ID writeups (iFixit has a series that explains common IC types like power management).
5. Make a quick list:
 - “Main SoC / application processor”
 - “Memory (RAM, storage)”
 - “PMIC”
 - “Wi-Fi/BT”
 - “Charging / USB controller”
 - “Audio codec”

What you're training: mapping *blocks* to *real-world functions*.

Even repair communities emphasize that identifying components often requires schematics/board context; visual guessing alone has limits – so using references matters. **d**